

0.48.2. Les réglages sous un nom : Carre — doublé, deux positions

Deux Carre, deux noms : ronde et coin.

1. Ce qu'il faut comprendre

C'est le 0.48 et le 0.48.1 réunis : deux `Carre`, chacun avec son nom.

Chaque nom range ses sept réglages : `carre(ronde)` et `carre(coin)` ne se mélangent pas.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2113 octets** de cartouche.

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
//          x   y  tuile      taille  sens  vitesse  tours
```

2. UNE VARIABLE — RONDE

```
Carre ronde = { 10,  8, ALPHABET[0], 2,      1,    250,    2 };
```

3. UNE VARIABLE — COIN

```
Carre coin  = {  4,  4, ALPHABET[1], 2,      1,    250,    2 };
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    carre(ronde);    // le A, autour de (10, 8)  
    carre(coin);     // puis le B, autour de (4, 4)  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

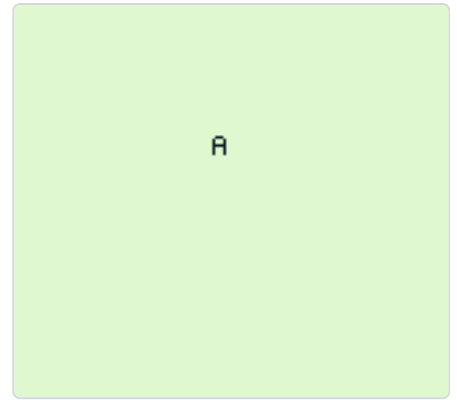
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



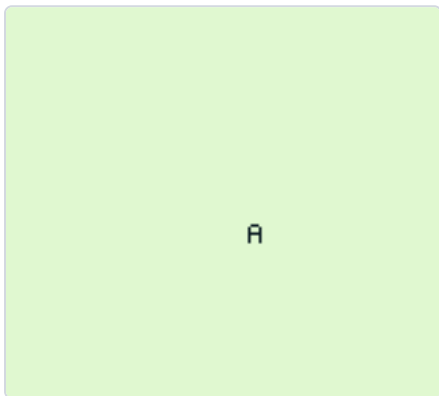
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Le A fait deux tours autour de (10, 8), puis le B autour de (4, 4).

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A finit en (10, 8) (10,8)

✓ le B finit en (4, 4) (4,4)

Ensuite : **0.49. De plus en plus grand : 1, puis 2** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++